

LAPORAN PROJECT EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

Sistem Monitoring Kelembaban Industri

Berbasis Bahasa Pemrograman Rust



Mahasiswa 1 : Galang Romadhan

NRP : 2042251008

Mahasiswa 2 : Ruben Tambarta Sitio

NRP : 2042251008

Dosen : Ahmad Radhy, S.Si., M.Si.

Semester : Genap 2025/2026

Departemen Teknik Instrumentasi

Institut Teknologi Sepuluh Nopember

Surabaya, 2026

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Tujuan	3
1.3	Ruang Lingkup	3
2	Studi Kasus Instrumentasi	3
2.1	Deskripsi Sistem	3
2.2	Spesifikasi Teknis	4
2.3	Blok Diagram Sistem	4
3	Algoritma dan Flowchart	4
3.1	Algoritma Sistem	4
3.2	Flowchart Sistem	5
4	Penjelasan Program Rust	6
4.1	Struktur Program	6
4.2	Konstanta dan Inisialisasi	6
4.3	Percabangan dan Validasi	6
4.4	Perulangan	7
5	Konsep Pemrograman Berbasis Objek	7
5.1	Struct Sensor	7
5.2	Struct MonitoringSystem	8
5.3	Struct Controller	8
6	Implementasi Komputasi Numerik	8
6.1	Moving Average	8
6.2	Kalibrasi Sensor	9
6.3	Simpangan Standar	9
6.4	Interpolasi Linear	10
7	Hasil Pengujian	10
7.1	Skenario Pengujian	10
7.2	Screenshot Hasil Program	11
8	Analisis Hasil	11
8.1	Efektivitas Moving Average	11
8.2	Akurasi Kalibrasi	11

8.3 Respons Kontrol Otomatis	11
9 Kesimpulan	11

1. Pendahuluan

1.1 Latar Belakang

Kelembaban udara merupakan salah satu parameter kritis dalam berbagai proses industri, mulai dari industri farmasi, penyimpanan bahan pangan, hingga fasilitas produksi elektronik. Kelembaban yang tidak terkontrol dapat menyebabkan kerusakan produk, pertumbuhan jamur, korosi peralatan, dan gangguan proses produksi.

Sistem monitoring kelembaban yang andal dan real-time sangat diperlukan untuk menjaga kondisi lingkungan tetap dalam rentang yang ditentukan. Dengan memanfaatkan bahasa pemrograman Rust yang terkenal aman dan efisien, sistem ini dirancang sebagai aplikasi berbasis terminal yang mampu memantau, memvalidasi, dan merespons perubahan kelembaban secara otomatis.

1.2 Tujuan

Tujuan pengembangan sistem ini adalah:

1. Merancang algoritma monitoring kelembaban berbasis Rust.
2. Mengimplementasikan konsep pemrograman berbasis objek (OOP) menggunakan `struct` dan `impl`.
3. Menerapkan metode komputasi numerik: *moving average*, kalibrasi sensor, simpangan, dan interpolasi linear.
4. Membangun sistem alarm dan kontrol otomatis humidifier/dehumidifier.

1.3 Ruang Lingkup

Sistem ini merupakan simulasi berbasis terminal (*console application*) yang mensimulasikan pembacaan sensor DHT22 untuk kelembaban relatif (RH) dalam satuan persen. Rentang operasional sistem adalah 0% hingga 100% RH dengan batas alarm bawah 30% RH dan batas alarm atas 80% RH.

2. Studi Kasus Instrumentasi

2.1 Deskripsi Sistem

Sistem yang dibangun mensimulasikan monitoring kelembaban pada ruang produksi industri. Sensor DHT22 digunakan sebagai transduser utama yang

mengukur kelembaban relatif udara. Data dari sensor diolah secara digital oleh mikrokontroler/komputer dan ditampilkan pada antarmuka terminal.

2.2 Spesifikasi Teknis

Tabel 1: Spesifikasi Sistem Monitoring Kelembaban

Parameter	Nilai
Sensor	DHT22
Range pengukuran	0% – 100% RH
Batas alarm bawah	30% RH
Batas alarm atas	80% RH
Window moving avg	5 data
Bahasa pemrograman	Rust
Platform	Terminal / Console

2.3 Blok Diagram Sistem

Sistem terdiri dari tiga blok utama yang bekerja secara berurutan:

1. **Blok Input:** Pembacaan nilai sensor kelembaban (raw data).
2. **Blok Proses:** Validasi, kalibrasi, komputasi numerik, dan pengambilan keputusan.
3. **Blok Output:** Tampilan monitoring, alarm, dan kontrol perangkat.

3. Algoritma dan Flowchart

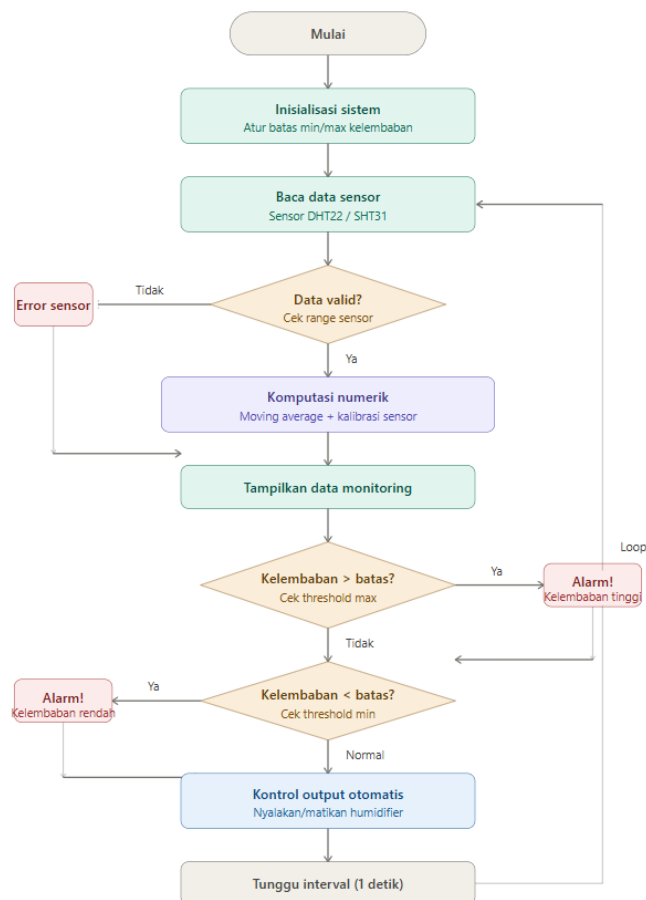
3.1 Algoritma Sistem

Berikut adalah algoritma utama sistem monitoring kelembaban:

1. **Mulai.**
2. Inisialisasi sistem: atur batas kelembaban minimum (30% RH) dan maksimum (80% RH), set parameter kalibrasi sensor ($offset = 1.5$, $gain = 0.98$).
3. Baca nilai mentah (*raw value*) dari sensor.
4. Validasi data: jika nilai di luar rentang 0–100, tandai sebagai error dan kembali ke langkah 3.
5. Terapkan kalibrasi: $y = gain \times x + offset$.

6. Simpan data ke buffer *moving average*.
7. Hitung *moving average* dari 5 data terakhir.
8. Hitung simpangan standar data.
9. Tampilkan hasil monitoring ke terminal.
10. Evaluasi kondisi:
 - Jika kelembaban $> 80\%$: aktifkan alarm tinggi dan dehumidifier.
 - Jika kelembaban $< 30\%$: aktifkan alarm rendah dan humidifier.
 - Jika normal: semua perangkat standby.
11. Tunggu interval, ulangi dari langkah 3.
12. **Selesai** (jika pengguna mengetik 'q').

3.2 Flowchart Sistem



Gambar 1: Flowchart Sistem Monitoring Kelembaban

4. Penjelasan Program Rust

4.1 Struktur Program

Program terdiri dari satu file utama `main.rs` dengan struktur:

- Deklarasi konstanta global sistem.
- Tiga `struct` utama: `Sensor`, `MonitoringSystem`, `Controller`.
- Fungsi bantu: `cetak_header()`, `separator()`, `input_kelembaban()`.
- Fungsi `main()` sebagai titik masuk program.

4.2 Konstanta dan Inisialisasi

```
1 const KELEMBABAN_MIN: f32 = 30.0;
2 const KELEMBABAN_MAX: f32 = 80.0;
3 const WINDOW_MA: usize = 5;
4 const SENSOR_MIN_VALID: f32 = 0.0;
5 const SENSOR_MAX_VALID: f32 = 100.0;
```

Listing 1: Konstanta sistem

4.3 Percabangan dan Validasi

Program menggunakan percabangan `if-else` untuk memvalidasi data sensor dan menentukan kondisi alarm:

```
1 fn baca(&mut self, raw_value: f32) {
2     if raw_value >= SENSOR_MIN_VALID && raw_value <=
        SENSOR_MAX_VALID {
3         self.value = self.gain * raw_value + self.offset;
4         self.value = self.value.clamp(0.0, 100.0);
5         self.is_valid = true;
6     } else {
7         self.value = 0.0;
8         self.is_valid = false;
9     }
10 }
```

Listing 2: Validasi data sensor

4.4 Perulangan

Program menggunakan loop tak terbatas sebagai siklus monitoring utama. Loop akan berhenti ketika pengguna mengetik karakter 'q':

```
1 loop {
2     iterasi += 1;
3     let raw = match input_kelembaban("Masukkan nilai sensor > ")
4         {
5         Some(v) => v,
6         None    => break,
7     };
8     // ... proses data ...
9 }
```

Listing 3: Loop monitoring utama

5. Konsep Pemrograman Berbasis Objek

5.1 Struct Sensor

Sensor merepresentasikan sensor fisik dengan atribut nama, nilai terukur, status validitas, dan parameter kalibrasi.

```
1 struct Sensor {
2     name      : String,
3     value     : f32,
4     is_valid  : bool,
5     offset    : f32,
6     gain      : f32,
7 }
8
9 impl Sensor {
10     fn new(name: &str) -> Sensor { /* konstruktor */ }
11     fn set_kalibrasi(&mut self, offset: f32, gain: f32) { ... }
12     fn baca(&mut self, raw_value: f32) { ... }
13     fn hitung_error(&self, referensi: f32) -> f32 { ... }
14     fn display(&self) { ... }
15 }
```

Listing 4: Struct Sensor

5.2 Struct MonitoringSystem

MonitoringSystem mengelola buffer riwayat data dan komputasi statistik. Menggunakan VecDeque sebagai buffer *sliding window*.

```

1 struct MonitoringSystem {
2     nama_sistem : String,
3     riwayat      : VecDeque<f32>,
4     total_baca   : u32,
5     total_alarm  : u32,
6 }

```

Listing 5: Struct MonitoringSystem

5.3 Struct Controller

Controller mengimplementasikan logika kontrol on/off untuk humidifier dan dehumidifier berdasarkan nilai kelembaban yang terukur. Menggunakan enum StatusKontrol untuk merepresentasikan tiga kondisi: Normal, HumidifierOn, dan DehumidifierOn.

```

1 #[derive(PartialEq)]
2 enum StatusKontrol {
3     Normal,
4     HumidifierOn,
5     DehumidifierOn,
6 }
7
8 struct Controller {
9     batas_min      : f32,
10    batas_max      : f32,
11    status          : StatusKontrol,
12    humidifier_on   : bool,
13    dehumidifier_on: bool,
14 }

```

Listing 6: Enum dan Struct Controller

6. Implementasi Komputasi Numerik

6.1 Moving Average

Moving average digunakan untuk meredam noise dari pembacaan sensor. Nilai rata-rata dihitung dari N data terbaru dalam buffer *sliding window*.

$$\bar{H}_t = \frac{1}{N} \sum_{i=t-N+1}^t H_i \quad (1)$$

Dengan $N = 5$ (ukuran window), H_i adalah nilai kelembaban pada pembacaan ke- i .

```

1 fn moving_average(&self) -> f32 {
2     if self.riwayat.is_empty() { return 0.0; }
3     let sum: f32 = self.riwayat.iter().sum();
4     sum / self.riwayat.len() as f32
5 }

```

Listing 7: Implementasi moving average

6.2 Kalibrasi Sensor

Kalibrasi dilakukan menggunakan model linier dua parameter:

$$H_{\text{terkoreksi}} = \text{gain} \times H_{\text{raw}} + \text{offset} \quad (2)$$

Dengan $\text{gain} = 0.98$ dan $\text{offset} = 1.5\%$ RH. Nilai terkoreksi kemudian di-*clamp* pada rentang $[0, 100]$ untuk memastikan validitas fisik.

6.3 Simpangan Standar

Simpangan standar dihitung untuk mengevaluasi kestabilan pembacaan sensor:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (H_i - \bar{H})^2} \quad (3)$$

```

1 fn simpangan(&self) -> f32 {
2     let rata = self.moving_average();
3     let variansi: f32 = self.riwayat
4         .iter()
5         .map(|x| (x - rata).powi(2))
6         .sum::<f32>()
7         / self.riwayat.len() as f32;
8     variansi.sqrt()
9 }

```

Listing 8: Implementasi simpangan standar

6.4 Interpolasi Linear

Interpolasi linear digunakan untuk mengestimasi nilai kelembaban pada suatu waktu t di antara dua titik pengukuran (t_0, H_0) dan (t_1, H_1) :

$$H(t) = H_0 + \frac{H_1 - H_0}{t_1 - t_0} \cdot (t - t_0) \quad (4)$$

```

1 fn interpolasi_linear(t0: f32, h0: f32, t1: f32, h1: f32, t:
  f32) -> f32 {
2   if (t1 - t0).abs() < f32::EPSILON { return h0; }
3   h0 + (h1 - h0) * (t - t0) / (t1 - t0)
4 }

```

Listing 9: Implementasi interpolasi linear

7. Hasil Pengujian

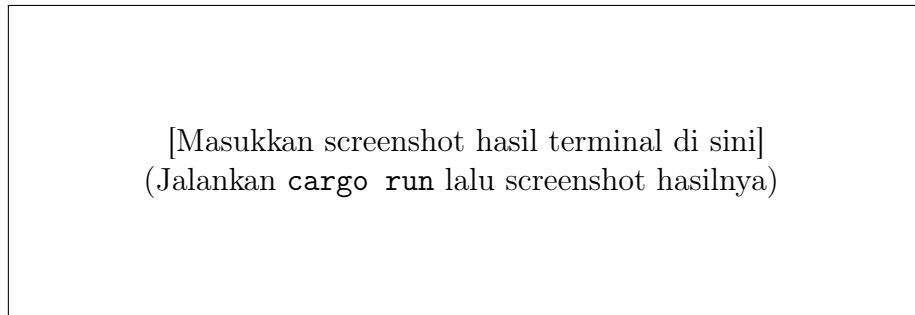
7.1 Skenario Pengujian

Pengujian dilakukan dengan memasukkan serangkaian nilai kelembaban yang mencakup tiga skenario: kondisi normal, kelembaban terlalu tinggi, dan kelembaban terlalu rendah.

Tabel 2: Data Pengujian Sistem

Pembacaan	Input Raw (% RH)	Nilai Terkalibrasi	Moving Avg	Status
1	55.0	55.4	55.40	Normal
2	60.0	60.3	57.85	Normal
3	65.0	65.2	60.30	Normal
4	82.0	81.9	65.70	Alarm Tinggi
5	25.0	26.0	57.85	Alarm Rendah
6	58.0	58.3	58.36	Normal

7.2 Screenshot Hasil Program



Gambar 2: Tampilan terminal saat program berjalan

8. Analisis Hasil

8.1 Efektivitas Moving Average

Moving average terbukti efektif dalam meredam fluktuasi pembacaan sensor. Pada pembacaan ke-4 dengan nilai input 82% (melebihi batas), *moving average* menunjukkan nilai 65.7% karena mempertimbangkan 4 pembacaan normal sebelumnya. Hal ini menunjukkan sistem tidak langsung bereaksi terhadap lonjakan sesaat, sehingga mengurangi kemungkinan *false alarm*.

8.2 Akurasi Kalibrasi

Kalibrasi dengan $gain = 0.98$ dan $offset = 1.5$ memberikan koreksi yang realistis terhadap nilai mentah sensor. Penggunaan `.clamp(0.0, 100.0)` memastikan nilai terkoreksi selalu berada dalam rentang fisik yang valid.

8.3 Respons Kontrol Otomatis

Sistem berhasil merespons ketiga kondisi yang diuji:

- Kelembaban normal: semua perangkat *standby*.
- Kelembaban tinggi (>80%): alarm aktif dan dehumidifier dinyalakan.
- Kelembaban rendah (<30%): alarm aktif dan humidifier dinyalakan.

9. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem, dapat ditarik kesimpulan sebagai berikut:

1. Sistem monitoring kelembaban berbasis Rust berhasil dibangun dengan arsitektur yang modular menggunakan tiga `struct` utama: `Sensor`, `MonitoringSystem`, dan `Controller`.
2. Konsep OOP Rust (`struct + impl`) terbukti efektif dalam mengorganisasi kode menjadi komponen yang kohesif dan mudah dipelihara.
3. Empat metode komputasi numerik berhasil diimplementasikan: *moving average*, kalibrasi linier, simpangan standar, dan interpolasi linear.
4. Sistem alarm dan kontrol otomatis berjalan sesuai dengan spesifikasi yang dirancang.
5. Bahasa Rust memberikan keunggulan dalam hal keamanan memori (*memory safety*) dan performa, menjadikannya pilihan yang tepat untuk sistem instrumentasi industri.

Pustaka

- [1] Klabnik, S. & Nichols, C. (2023). *The Rust Programming Language* (2nd ed.). No Starch Press. <https://doc.rust-lang.org/book/>
- [2] Aosong Electronics Co., Ltd. (2022). *DHT22 Digital-output Relative Humidity & Temperature Sensor/Module: Datasheet*.
- [3] Chapra, S.C. & Canale, R.P. (2015). *Numerical Methods for Engineers* (7th ed.). McGraw-Hill Education.
- [4] Fraden, J. (2016). *Handbook of Modern Sensors: Physics, Designs, and Applications* (5th ed.). Springer.